Mobiloca User Service

Documentation de Déploiement Docker

Microservice : mobiloca-user-service
Port : 2021
Profil Spring : local
Java : 17 (Eclipse Temurin)
Framework : Spring Boot

Izemx

Date : 11 février 2026

Version : 0.0.1-SNAPSHOT

Table des matières

1 Présentation du Microservice **2**
1.1 Informations Générales . . . 2
1.2 Dépendances Principales . . . 2
1.3 Fonctionnalités Principales . . . 2

2 Prérequis **3**
2.1 Logiciels Requis . . . 3
2.2 Identifiants Nexus . . . 3
2.3 Accès Réseau . . . 3

3 Architecture Docker **3**
3.1 Stratégie de Build Multi-Stage . . . 3
3.2 Contexte de Build . . . 4

4 Fichiers de Configuration Docker **4**
4.1 Dockerfile . . . 4
4.2 Docker Compose . . . 5

5 Procédure de Déploiement **5**
5.1 Étape 1 : Naviguer vers le répertoire du service . . . 5
5.2 Étape 2 : Définir les variables d'environnement et lancer . . . 5
5.3 Étape 3 : Vérifier le déploiement . . . 6
5.4 Étape 4 : Accéder au service . . . 6

6 Commandes Utiles **6**

7 Problèmes Rencontrés et Solutions **6**
7.1 Problème 1 : Image openjdk :17-jdk-slim introuvable . . . 6
7.2 Problème 2 : mobiloca-common introuvable sur Nexus . . . 7

8 Configuration de l'Application **7**
8.1 Profil Local (application-local.yml) . . . 7
8.2 Sécurité et Authentification . . . 7
8.3 Services Externes Référencés . . . 7

Présentation du Microservice

Le microservice **mobiloca-user-service** est un composant central de l'architecture microservices du projet Mobiloca. Il est responsable de la **gestion des utilisateurs**, de l'**authentification**, et de la **gestion des profils** des clients et administrateurs de la plateforme.

Informations Générales

Propriété	Valeur
Nom du service	mobiloca-user-service
GroupId Maven	com.marketingconfort
ArtifactId	mobiloca-user-service
Version	0.0.1-SNAPSHOT
Version Java	17
Port du serveur	2021
Profil par défaut	local
Base de données	PostgreSQL (distante)
Authentification	Keycloak (OAuth2 / JWT)

Table 1 – Informations générales du microservice

Dépendances Principales

- **mobiloca-common** (v0.0.70-SNAPSHOT) – Bibliothèque commune partagée
- **mc-starter-parent** (v1.0.68-RELEASE) – Parent POM de Marketing Confort
- **Spring Boot Starter Data JPA** – Persistance des données
- **Apache HttpClient 5** (v5.3.1) – Client HTTP
- **Spring Security OAuth2** – Authentification JWT via Keycloak
- **Lombok** – Réduction du code boilerplate

Fonctionnalités Principales

- Gestion des comptes utilisateurs (CRUD)
- Authentification et autorisation via Keycloak
- Gestion des profils utilisateurs
- Upload de fichiers (photos de profil, documents)
- Envoi d'emails (SMTP via Titan Email)
- Stockage de fichiers sur AWS S3
- Intégration avec le service d'agences et le service de véhicules

Prérequis

Avant de procéder au déploiement, assurez-vous que les éléments suivants sont installés et configurés :

Logiciels Requis

Logiciel	Version minimale	Vérification
Docker	20.0+	`docker -version`
Docker Compose	2.0+	`docker compose version`

Table 2 – Logiciels requis

Identifiants Nexus

Les identifiants du repository Maven Nexus sont nécessaires pour télécharger les dépendances privées depuis `repo.mc-git.com` :

- **NEXUS_USER** – Nom d'utilisateur Nexus
- **NEXUS_PASSWORD** – Mot de passe Nexus

Accès Réseau

- Accès au repository Nexus : `https://repo.mc-git.com/repository/maven-group/`
- Accès à la base de données PostgreSQL distante : `195.35.24.133:5432`
- Accès au serveur Keycloak : `https://keycloak-rec.mc-intern.com/auth/`
- Accès au Docker Hub pour les images de base

Architecture Docker

Stratégie de Build Multi-Stage

Le Dockerfile utilise une approche **multi-stage build** en deux étapes :

1. **Étape de Build** (`maven:3.8-eclipse-temurin-17`) :
   - Configuration des identifiants Nexus dans `settings.xml`
   - Compilation et installation de `mobiloca-common` dans le repository Maven local
   - Compilation et packaging de `mobiloca-user-service`
2. **Étape d'Exécution** (`eclipse-temurin:17-jre-jammy`) :
   - Image légère avec uniquement le JRE
   - Copie du JAR généré depuis l'étape de build
   - Exposition du port 2021

Note importante

Les images Docker `openjdk:17-jdk-slim` et `maven:3.8-openjdk-17-slim` ont été **dépréciées** et retirées de Docker Hub. Elles ont été remplacées par `eclipse-temurin:17-jre-jammy` et `maven:3.8-eclipse-temurin-17` respectivement.

Contexte de Build

Le contexte de build est défini au niveau du **répertoire parent** (`mobiloca-project/`) afin de pouvoir accéder à la fois au code de `mobiloca-common` et de `mobiloca-user-service` lors du build Docker.

Fichiers de Configuration Docker

Dockerfile

```
ARG ARCH=linux/amd64

# Etape de build
FROM maven:3.8-eclipse-temurin-17 AS build

ARG NEXUS_USER
ARG NEXUS_PASSWORD

WORKDIR /app

# Creer le repertoire pour les settings Maven
RUN mkdir -p /root/.m2

# Copier le fichier settings.xml et remplacer les identifiants
COPY mobiloca-user-service/scripts/settings.xml
   /root/.m2/settings.xml
RUN sed -i "s/NEXUS_USER/${NEXUS_USER}/g"
   /root/.m2/settings.xml && \
    sed -i "s/NEXUS_PASSWORD/${NEXUS_PASSWORD}/g"
       /root/.m2/settings.xml

# D'abord, builder et installer mobiloca-common
COPY mobiloca-common/pom.xml /app/mobiloca-common/pom.xml
COPY mobiloca-common/src /app/mobiloca-common/src
RUN cd /app/mobiloca-common && mvn clean install -DskipTests

# Ensuite, builder le service user
COPY mobiloca-user-service/pom.xml .
COPY mobiloca-user-service/src ./src
RUN mvn clean package -DskipTests

# Etape d'execution
```

```
FROM --platform=${ARCH} eclipse-temurin:17-jre-jammy
COPY --from=build /app/target/*.jar /app.jar
EXPOSE 2021
ENV SPRING_PROFILES_ACTIVE=local
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

Listing 1 – Dockerfile – mobiloca-user-service

Docker Compose

```
services:
  mobiloca-user-service:
    build:
      context: ..
      dockerfile: mobiloca-user-service/Dockerfile
      args:
        NEXUS_USER: ${NEXUS_USER}
        NEXUS_PASSWORD: ${NEXUS_PASSWORD}
    container_name: mobiloca-user-service
    ports:
      - "2021:2021"
    environment:
      - SPRING_PROFILES_ACTIVE=local
    restart: unless-stopped
```

Listing 2 – docker-compose.yml – mobiloca-user-service

Procédure de Déploiement

Étape 1 : Naviguer vers le répertoire du service

```
cd mobiloca-project/mobiloca-user-service
```

Listing 3 – Navigation vers le répertoire

Étape 2 : Définir les variables d'environnement et lancer

```
$env:NEXUS_USER="votre_utilisateur_nexus"
$env:NEXUS_PASSWORD="votre_mot_de_passe_nexus"
docker-compose up --build -d
```

Listing 4 – Lancement avec Docker Compose (PowerShell)

Ou en une seule commande :

```
$env:NEXUS_USER="votre_utilisateur"; ‘
```

```
$env:NEXUS_PASSWORD="votre_mdp"; `
docker-compose up --build -d
```

Listing 5 – Commande en une ligne (PowerShell)

Sous Linux / macOS

```
NEXUS_USER=votre_utilisateur \
NEXUS_PASSWORD=votre_mdp \
docker-compose up --build -d
```

Étape 3 : Vérifier le déploiement

```
# Verifier que le container est en cours d'execution
docker ps --filter "name=mobiloca-user-service"

# Consulter les logs
docker logs mobiloca-user-service --tail 30

# Verifier l'acces HTTP
curl http://localhost:2021
```

Listing 6 – Vérification du container

Étape 4 : Accéder au service

Ouvrir un navigateur et accéder à :

`http://localhost:2021`

Le service affiche un formulaire d'authentification Spring Security. L'accès aux endpoints REST nécessite une authentification via JWT (Keycloak).

Commandes Utiles

Problèmes Rencontrés et Solutions

Problème 1 : Image openjdk :17-jdk-slim introuvable

Erreur : `docker.io/library/openjdk:17-jdk-slim: not found`

Cause : L'image `openjdk:17-jdk-slim` a été dépréciée et retirée de Docker Hub.

Solution : Remplacer par `eclipse-temurin:17-jre-jammy` pour l'exécution et `maven:3.8-eclipse-temurin-17` pour le build.

Action	Commande
Démarrer le service	`docker-compose up -d`
Arrêter le service	`docker-compose down`
Reconstruire et démarrer	`docker-compose up -build -d`
Voir les logs en temps réel	`docker logs -f mobiloca-user-service`
Voir le statut	`docker ps`
Redémarrer le container	`docker restart mobiloca-user-service`
Accéder au shell du container	`docker exec -it mobiloca-user-service bash`
Supprimer le container	`docker-compose down -rmi all`

Table 3 – Commandes Docker utiles

Problème 2 : mobiloca-common introuvable sur Nexus

Erreur : `Could not find artifact com.marketingconfort:mobiloca-common:jar:0.0.70-SNA`

Cause : Le SNAPSHOT de mobiloca-common n'est pas publié sur le repository Nexus.

Solution : Modifier le Dockerfile pour compiler et installer `mobiloca-common` localement avant de compiler le service. Le contexte de build a été élargi au répertoire parent pour accéder au code source de mobiloca-common.

Configuration de l'Application

Profil Local (application-local.yml)

Sécurité et Authentification

Le service utilise **Spring Security avec OAuth2 Resource Server** pour la validation des tokens JWT émis par Keycloak.

- **Type d'authentification :** OAuth2 / JWT
- **Issuer URI :** `https://keycloak-rec.mc-intern.com/auth/realms/mobiloca-local`
- **Client ID :** `mobiloca-backend-client`
- L'accès depuis le navigateur affiche un formulaire de connexion Spring Security par défaut
- L'accès programmatique nécessite un token JWT valide dans l'en-tête `Authorization: Bearer <token>`

Services Externes Référencés

Propriété	Valeur
Base de données	PostgreSQL sur 195.35.24.133 :5432
Nom de la BDD	mobiloca-user-service-local-db
JPA ddl-auto	update
Keycloak URL	https ://keycloak-rec.mc-intern.com/auth/
Keycloak Realm	mobiloca-local
Client ID	mobiloca-backend-client
OAuth2 JWT Issuer	https ://keycloak-rec.mc-intern.com/auth/realms/mobiloca-local
AWS Bucket	mobiloca-rec
Serveur SMTP	smtp.titan.email (port 587)
Email expéditeur	test@izemx.com
Max upload images	22MB
Max upload vidéos	500MB
Max upload documents	10MB
Cache	EhCache (local-ehcache.xml)

Table 4 – Configuration du profil local

Service	**Configuration**
Agency Service	`external-api.agency-service-url`
Vehicle Service	`external-api.vehicle-service-url`

Table 5 – Services externes référencés dans la configuration